Universidad Autónoma de Baja California

Facultad de Ciencias de la Ingeniería y Tecnología

Estructuras de Datos

Sistema de Gestión de Atención Médica - Práctica Unidad 3

Integrantes:

- David Osvaldo Calderón Salgado
- Absalón Castellano Flores
- Miguel Ángel Lara Tarín
- Jared Alejandro López Aguilera

Docente: José Manuel Muñoz Contreras

Código de la Materia: 40007

16 de marzo de 2026

Introducción

Este proyecto consiste en el desarrollo de un sistema para la gestión de flujos de pacientes en un entorno hospitalario. El objetivo principal es optimizar la atención mediante la implementación de colas de prioridad, permitiendo el registro, atención y seguimiento de pacientes en tiempo real a través de una interfaz moderna desarrollada en Python con customtkinter.

Descripción de la Estructura de Datos

Para este sistema se seleccionaron Deques (Colas de doble extremo) y una Lógica de Prioridad. Esta elección se fundamenta en:

- **Gestión FIFO:** Garantiza que los pacientes regulares sean atendidos en el orden exacto de su llegada.

- **Priorización Médica:** Permite que los casos de "Emergencia" (Prioridad 1) se posicionen al frente de la atención, independientemente de cuántos pacientes regulares haya en espera.

Arquitectura del Sistema

El software se divide en cuatro componentes fundamentales:

- **Clase Paciente:** Almacena los datos del usuario, su prioridad, departamento y sellos de tiempo para cálculos de eficiencia.

- **Clase ColaPrioridad:** El motor de estructuras que gestiona dos deques internos (emergencias y regulares) para el control de turnos.

- **Clase GestionHospital:** Actúa como el cerebro del sistema, manejando la persistencia en archivos JSON y las validaciones de negocio.

- **Clase AppMedica:** Proporciona la interfaz gráfica y el sistema de verificación automática de reservas.

Decisiones de Diseño y Validaciones

Para garantizar que el sistema sea confiable y fácil de usar, se tomaron las siguientes decisiones técnicas:

- **Separación de Capas:** Se mantuvo una independencia total entre la lógica del hospital y la GUI, facilitando el mantenimiento y la escalabilidad del sistema.
- **Verificador Automático de Reservas:** Se implementó un hilo de ejecución secundario en la interfaz que consulta cada 10 segundos las citas futuras, activándolas automáticamente al llegar la hora pactada.
- **Persistencia Robusta:** Los datos se almacenan en una estructura JSON organizada por departamentos, asegurando que la información no se pierda al cerrar la aplicación.
- **Validación de Integridad:** El sistema rechaza nombres con caracteres numéricos y bloquea el agendamiento de citas en horarios que ya han transcurrido.

Casos de Prueba Ejecutados

El sistema fue validado mediante Unit Tests, obteniendo un resultado de 7 pruebas exitosas:

1. **Prioridad de Emergencia:** Confirmación de que pacientes P1 salen antes que P2.
2. **Orden FIFO:** Validación de que a igual prioridad, el primero en llegar es el primero en salir.
3. **Filtro de Nombres:** Rechazo exitoso de entradas de texto inválidas.
4. **Agendamiento Futuro:** Verificación dinámica de que las reservas solo se permitan para horarios posteriores al actual.
5. **Cálculo de Eficiencia:** Validación de que el tiempo de espera promedio sea mayor a cero tras una atención.

Diagrama UML

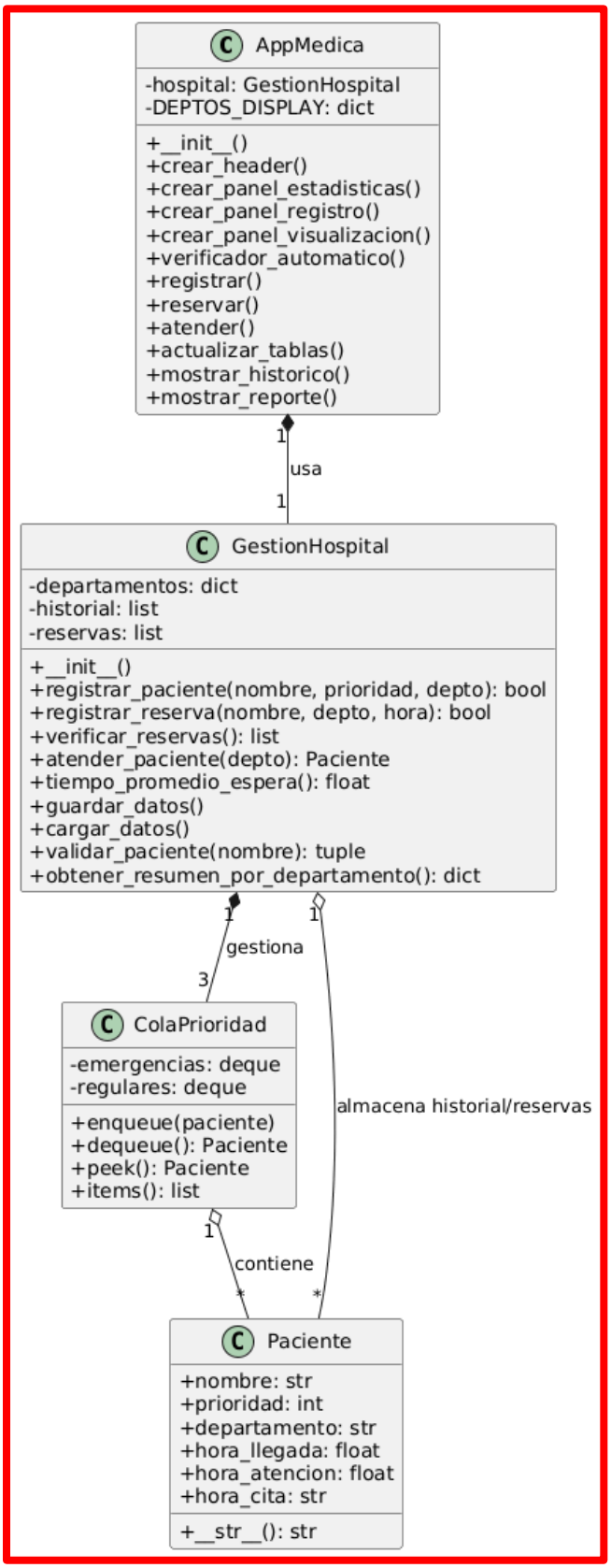
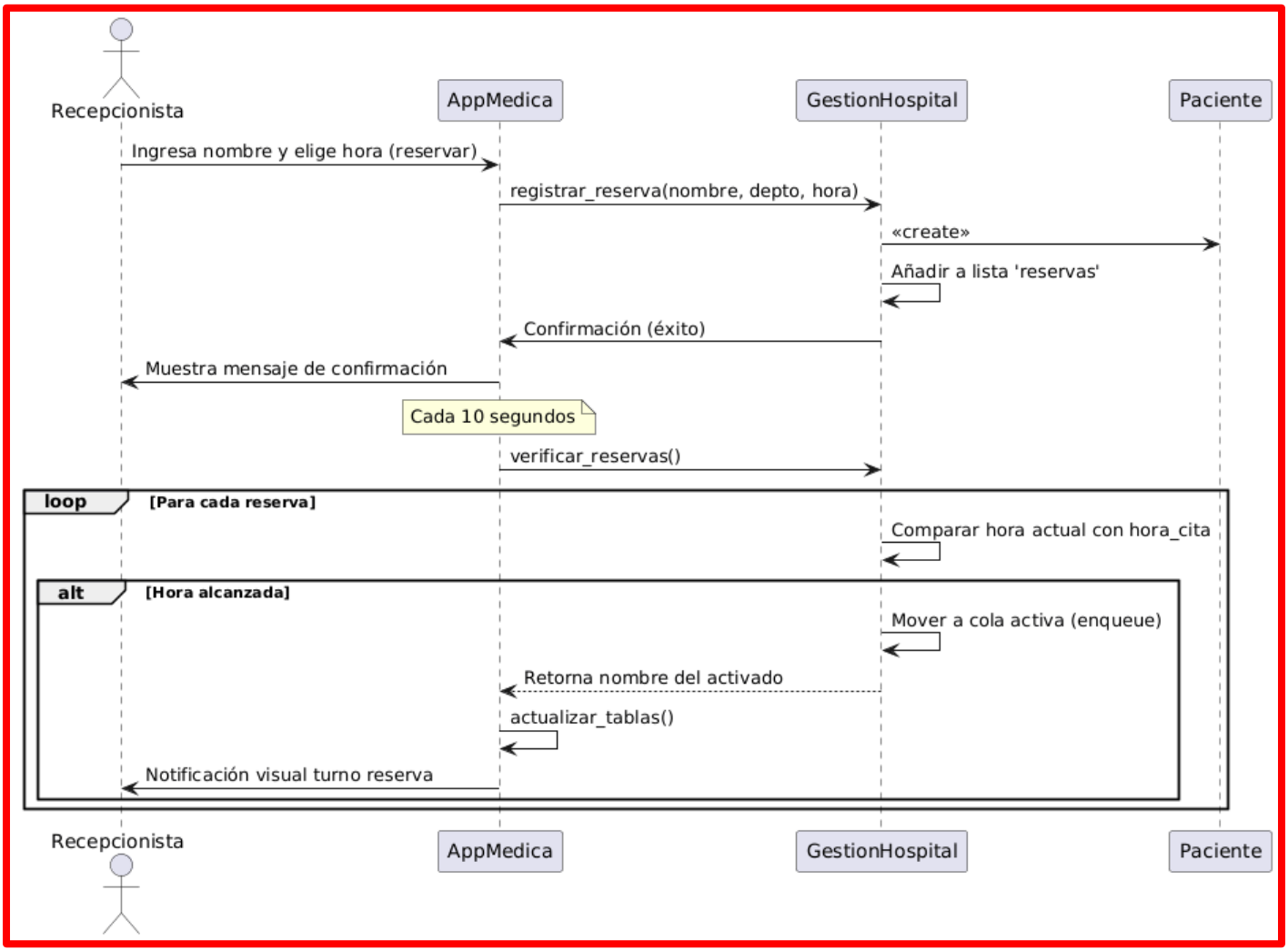


Diagrama de Secuencia



Resultado Final: Interfaz del Sistema

La interfaz gráfica final presenta un diseño organizado en tres paneles principales:

- **Panel de Estadísticas (Izquierda):** Muestra métricas de desempeño en tiempo real, como el total de pacientes atendidos y el tiempo de espera promedio global. Incluye botones para consultar el Histórico de Atenciones en una ventana emergente y para Generar el Reporte de eficiencia físico.
- **Gestión de Ingreso (Centro):** Permite realizar dos tipos de operaciones críticas: el ingreso inmediato mediante un sistema de prioridades (Emergencia/Regular) y el agendamiento de Reservas mediante selectores de tiempo.
- **Sala de Espera (Derecha):** Utiliza un sistema de tarjetas dinámicas con código de colores para identificar rápidamente las emergencias (color rojo) de los pacientes regulares o con cita (color gris azulado). Los encabezados de cada departamento están centrados para mejorar la jerarquía visual.

El resultado es un programa intuitivo que no solo reacciona a la llegada de pacientes, sino que gestiona citas futuras y automatiza las notificaciones de turno.

Conclusión

El sistema entregado cumple satisfactoriamente con todos los requerimientos obligatorios y las cuatro mejoras opcionales sugeridas: Notificaciones mediante cuadros de mensaje, Reservas de citas futuras, una Interfaz Gráfica intuitiva y un Histórico detallado para análisis posteriores.

La implementación de este proyecto demostró que el uso correcto de estructuras de datos lineales y una arquitectura modular permite desarrollar herramientas de software profesionales y confiables para problemas del mundo real.